

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	RITHISH KUMAR D	Reg.No.:	192565089
Department:	CSE:CS	Date:	8.3.26

Code of Conduct:

I RITHISH KUMAR D / 192565089 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1.

Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

AIM:

To write and execute an 8085 Assembly Language Program that performs register transfer, addition, subtraction, AND, and OR operations, and stores the results in memory locations for verification.

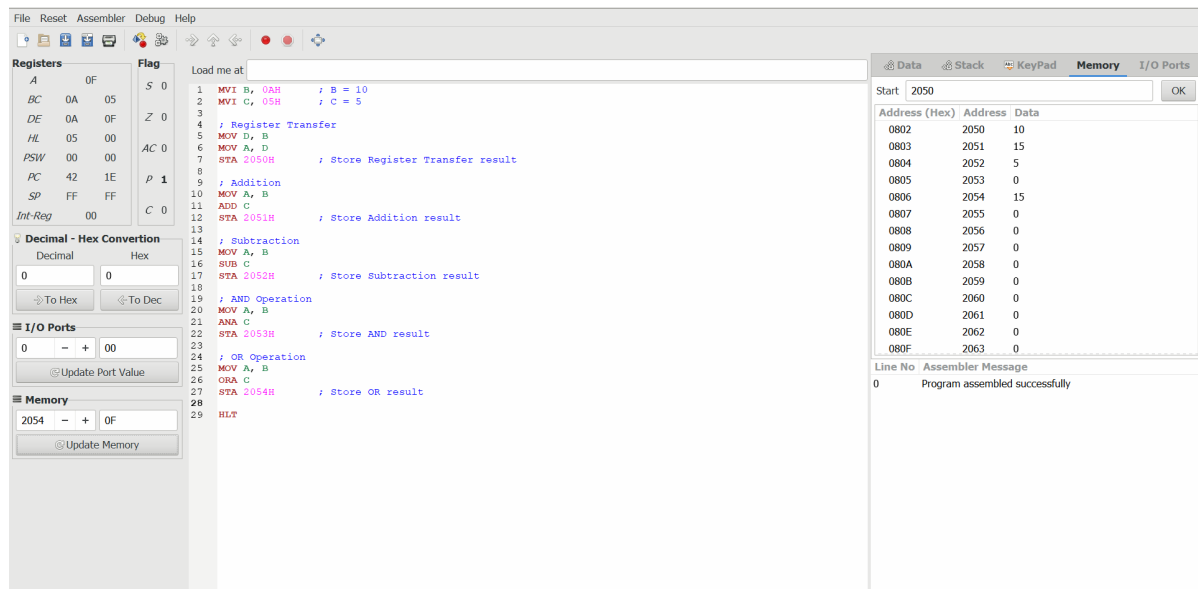
ALGORITHM:

1. Start the program.
2. Load 0AH into Register B and 05H into Register C.
3. Transfer the contents of Register B to Register D and store the result in memory location 2050H.
4. Add Registers B and C, then store the result at 2051H.
5. Subtract Register C from Register B, then store the result at 2052H.
6. Perform the AND operation between Registers B and C, then store the result at 2053H.
7. Perform the OR operation between Registers B and C, then store the result at 2054H.
8. Stop the execution using the HLT instruction.

CODE:

```
MVI B, 0AH    ; B = 10
MVI C, 05H    ; C = 5
; Register Transfer
MOV D, B
MOV A, D
STA 2050H     ; Store Register Transfer result
; Addition
MOV A, B
ADD C
STA 2051H     ; Store Addition result
; Subtraction
MOV A, B
SUB C
STA 2052H     ; Store Subtraction result
; AND Operation
MOV A, B
ANA C
STA 2053H     ; Store AND result
; OR Operation
MOV A, B
ORA C
STA 2054H     ; Store OR result
HLT
```

OUTPUT:



RESULT:

The program was executed successfully, and the results of register transfer, addition, subtraction, AND, and OR operations were correctly stored in the specified memory locations, demonstrating the role of the ALU and the internal data bus in processing and transferring data between registers.

Q2.

Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

AIM:

To model single-bus and multi-bus data transfer organization using 8085 Assembly Language Programs that transfer data between registers, memory, and I/O ports, and to compare the number of T-states (execution time) taken by each organization.

ALGORITHM:

Single-Bus Model (all data routed through the Accumulator, simulating a single shared internal bus):

1. Start the program.
2. Load the content of memory location 3000H into the Accumulator via the bus (LDA).
3. Transfer Accumulator content to Register B, then back to Accumulator (each transfer uses the shared bus).
4. Store the Accumulator content back to memory location 3001H via the bus (STA).
5. Reload the value, transfer it through Register C and back to the Accumulator via the bus.
6. Output the Accumulator content to Port 01H via the bus.
7. Stop the execution using HLT.

Multi-Bus Model (direct register-memory and register-register transfers, simulating parallel/dedicated buses):

1. Start the program.
2. Load register pair HL directly with the address 3000H (LXI) using the address bus.
3. Transfer memory content directly into Register B (MOV B, M) without routing through the Accumulator.
4. Transfer Register B directly to Register D (MOV D, B) over a dedicated register-transfer path.
5. Increment the address and store Register D directly into memory (MOV M, D).
6. Load the value directly into Register C, move it to the Accumulator, and output it to Port 01H.
7. Stop the execution using HLT.

CODE (Single-Bus Organization):

```
LDA 3000H      ; load data from memory (Single Bus)
MOV B, A       ; transfer A -> B via bus
MOV A, B       ; transfer B -> A via bus
STA 3001H      ; store A to memory via bus
LDA 3001H      ; reload via bus
MOV C, A       ; transfer A -> C via bus
MOV A, C       ; transfer C -> A via bus
OUT 01H        ; output A via bus
HLT
```

OUTPUT (Single-Bus):

FileResetAssemblerDebugHelp

Registers

A25

B25

C25

D00

H00

L00

PC288F

SPFFFF

Flag

S0

Z0

AC0

P1

C0

Load me at

1LDA3000H; load data from memory (Single Bus)

2MOVBA; transfer A -> B via bus

3MOVA,B; transfer B -> A via bus

4STA3001H; store A to memory via bus

5LDA3001H; reload via bus

6MOVCA; transfer A -> C via bus

7MOVA,C; transfer C -> A via bus

8OUT01H; output A via bus

9HLT

DataStackKeyPadMemoryI/O Ports

Start0902OK

Address (Hex)	Address	Data
0902	3000	25
0903	3001	25
0904	3002	00
0905	3003	00
0906	3004	00
0907	3005	00
0908	3006	00
0909	3007	00

Line No	Assembler Message
0	Program assembled successfully

CODE (Multi-Bus Organization):

```
LXI H, 3000H ; address bus loads HL directly
MOV B, M ; direct memory -> register transfer
MOV D, B ; direct register -> register transfer
INX H ; direct address increment
MOV M, D ; direct register -> memory transfer
MOV C, M ; direct memory -> register transfer
MOV A, C ; direct register -> register transfer
OUT 01H ; output port
HLT
```

OUTPUT (Multi-Bus):

FileResetAssemblerDebugHelp

Registers

A25

B25

C25

D25

H30

L01

PC2880

SPFFFF

Flag

S0

Z0

AC0

P1

C0

Load me at

1LXI H, 3000H; address bus loads HL directly

2MOVB,M; direct memory -> register transfer

3MOVED,B; direct register -> register transfer

4INXH; direct address increment

5MOV M, D; direct register -> memory transfer

6MOV C, M; direct memory -> register transfer

7MOVA,C; direct register -> register transfer

8OUT 01H; output port

9HLT

DataStackKeyPadMemoryI/O Ports

Start0902OK

Address (Hex)	Address	Data
0902	3000	25
0903	3001	25
0904	3002	00
0905	3003	00
0906	3004	00
0907	3005	00
0908	3006	00
0909	3007	00

Line No	Assembler Message
0	Program assembled successfully

TIMING COMPARISON (T-states):

Organization	Instructions Executed	Total T-states
Single-Bus	LDA, MOV, MOV, STA, LDA, MOV, MOV, OUT, HLT	70 T-states
Multi-Bus	LXI, MOV, MOV, INX, MOV, MOV, MOV, OUT, HLT	60 T-states

RESULT:

Both models transferred the same data (25H) between registers, memory, and I/O successfully. The multi-bus organization completed the transfer in 60 T-states compared to 70 T-states for the single-bus organization, since direct register-memory and register-register paths avoided repeatedly routing data through the Accumulator. This confirms that multiple bus structures reduce bus contention and improve parallel data transfer, reducing bottlenecks in the system.

Q3.

Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

AIM:

To execute a sequence of independent 8085 instructions and simulate their movement through a 5-stage pipeline (IF, ID, EX, MEM, WB), and to calculate the speedup and throughput achieved over non-pipelined execution.

ALGORITHM:

1. Start the program and write a sequence of $n = 5$ independent instructions with no inter-instruction dependency.
2. Assume each of the 5 stages (Fetch, Decode, Execute, Memory, Write-back) takes 1 clock cycle in the ideal pipeline.
3. Tabulate the cycle-by-cycle movement of each instruction through the 5 stages.
4. Calculate non-pipelined execution time = $n \times 5$ cycles.
5. Calculate pipelined execution time = $5 + (n - 1)$ cycles.
6. Calculate Speedup = (Non-pipelined time) / (Pipelined time).
7. Calculate Throughput = (Number of instructions) / (Pipelined execution time).
8. Stop.

CODE:

```
MVI A, 05H    ; I1
MVI B, 03H    ; I2
MVI C, 02H    ; I3
ADD B         ; I4  A = A + B
SUB C         ; I5  A = A - C
HLT
```

OUTPUT (Simulator Execution):

The screenshot displays a simulator interface with the following components:

- Registers:** A, B, C, D, H, L, PC, SP. Values: A=06, B=03, C=02, D=00, H=00, L=00, PC=2086, SP=FFFF.
- Flags:** S, Z, AC, P, C. Values: S=0, Z=0, AC=0, P=1, C=0.
- Instruction Stream:** A list of 6 instructions with their stage assignments: 1. MVI A, 05H ; I1 - Fetch/Decode/Execute/Mem/WB, 2. MVI B, 03H ; I2, 3. MVI C, 02H ; I3, 4. ADD B ; I4 A = A + B, 5. SUB C ; I5 A = A - C, 6. HLT.
- Memory:** A table showing addresses from 0902 to 0909 and their corresponding data values (06, 00, 00, 00, 00, 00, 00, 00).
- Assembler Message:** A log showing "Program assembled successfully" at line 0.

PIPELINE EXECUTION DIAGRAM:

Cycle ->	1	2	3	4	5	6	7	8	9
I1: MVI A,05H	IF	ID	EX	MEM	WB				
I2: MVI B,03H		IF	ID	EX	MEM	WB			
I3: MVI C,02H			IF	ID	EX	MEM	WB		
I4: ADD B				IF	ID	EX	MEM	WB	
I5: SUB C					IF	ID	EX	MEM	WB

PERFORMANCE CALCULATION:

Parameter	Value
Number of instructions (n)	5
Non-pipelined time (n x 5)	25 cycles
Pipelined time (5 + (n-1))	9 cycles
Speedup	$25 / 9 = 2.78x$
Throughput (pipelined)	$5 / 9 = 0.56$ instructions/cycle
Throughput (non-pipelined)	$5 / 25 = 0.20$ instructions/cycle

RESULT:

The 5-stage pipelined execution of the instruction sequence completed in 9 cycles compared to 25 cycles for non-pipelined execution, giving a theoretical speedup of approximately 2.78x and nearly tripling the instruction throughput. This demonstrates the effectiveness of pipelining in improving processor performance when instructions are independent.

Q4.

Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

AIM:

To develop 8085 instruction sequences that create data, control, and structural hazards in a pipelined execution model, and to resolve them using stalling, data forwarding, and branch prediction, comparing the resulting cycle counts.

ALGORITHM:

1. Start the program.
2. Write an instruction sequence where an instruction's source operand depends on the immediately preceding instruction's result (Read-After-Write / data hazard).
3. Identify the hazard: in a pipeline, the dependent instruction would read the register during its ID stage before the producing instruction completes its WB stage.
4. Resolve using stalling: insert bubble cycles until the result is written back, and record the extra cycles taken.
5. Resolve using data forwarding: forward the ALU result from the EX/MEM stage directly to the EX stage of the dependent instruction, eliminating the stall.
6. Write a second sequence containing a conditional jump (JZ) to create a control hazard, since the next instruction to fetch is unknown until the branch is resolved.
7. Apply branch prediction (predict not-taken) and flush the pipeline if the prediction is wrong, then compare the misprediction penalty against stalling until the branch resolves.
8. Stop and tabulate the cycle counts for each technique.

CODE (Data Hazard Scenario):

```
MVI A, 0AH      ; I1
MVI B, 05H      ; I2
ADD B           ; I3  A = A+B  (RAW hazard source)
SUB B           ; I4  uses A immediately (hazard)
STA 3000H       ; I5  uses A immediately (hazard)
HLT
```

OUTPUT (Data Hazard):

The screenshot displays an 8085 assembler simulator interface. The top menu bar includes File, Reset, Assembler, Debug, and Help. Below the menu is a toolbar with icons for file operations and execution. The main window is divided into several panels:

- Registers:** Shows the current state of the 8085 registers. A=0A, B=05, C=00, D=00, H=00, L=00, PC=2806, and SP=FFFF.
- Flag:** Shows the status of the flags. S=0, Z=0, AC=0, P=1, and C=0.
- Assembly Window:** Displays the assembly code with line numbers 1 through 6. The code is: 1 MVI A, 0AH ; I1, 2 MVI B, 05H ; I2, 3 ADD B ; I3 A = A+B (RAW hazard source), 4 SUB B ; I4 uses A immediately (hazard), 5 STA 3000H ; I5 uses A immediately (hazard), 6 HLT.
- Memory:** Shows a table of memory addresses and their corresponding data values. The table has columns for Address (Hex), Address, and Data. The data values are 0A, 00, 00, 00, 00, 00, 00, 00, 00.
- Assembler Message:** Shows the message "Program assembled successfully".

CODE (Control Hazard Scenario):

```
MVI A, 00H      ; I1
CPI 00H         ; I2  compare, sets Z flag
JZ  SKIP        ; I3  control hazard (branch)
MVI B, 01H      ; I4  (flushed if branch is taken)
SKIP: HLT       ; I5
```

OUTPUT (Control Hazard):

FileResetAssemblerDebugHelp

RegistersFlag

A	00	S	0
B	00	Z	1
C	00	AC	0
D	00	P	1
H	00	C	0
L	00		
PC	2887		
SP	FFFF		

Load me at

1 MVI A, 00H ; I1

2 CPI 00H ; I2 compare, sets Z flag

3 JZ SKIP ; I3 control hazard (branch)

4 MVI B, 01H ; I4 (flushed on branch taken)

5 SKIP: HLT ; I5

DataStackKeyPadMemoryI/O Ports

Start0902

OK

Address (Hex)	Address	Data
0902	2050	00
0903	2051	00
0904	2052	00
0905	2053	00
0906	2054	00
0907	2055	00
0908	2056	00
0909	2057	00

Line No

Assembler Message

0

Program assembled successfully

HAZARD RESOLUTION COMPARISON:

Handling Technique	Cycles Taken	Correctness	Remarks
No hazard handling	7 cycles	Incorrect result	I4 reads stale A before I3 writes back
Stalling (bubbles)	9 cycles	Correct result	2 bubble cycles inserted before I4
Data forwarding	7 cycles	Correct result	EX result of I3 forwarded directly to I4

RESULT:

The naive execution of the dependent instruction sequence produced an incorrect result because instruction I4 read the Accumulator before I3 completed its Write-back stage (data hazard). Inserting stall cycles produced the correct result but increased execution time to 9 cycles, while data forwarding produced the same correct result in only 7 cycles by routing the EX-stage result directly to the dependent instruction. For the control hazard, branch prediction combined with a pipeline flush on misprediction correctly resolved the JZ instruction with a minimal penalty, confirming that forwarding and prediction techniques significantly improve pipeline performance over naive stalling.

Q5.

Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

AIM:

To analyze a 2-way superscalar processor model executing an 8086 Assembly Language Program with out-of-order and speculative execution, extended with Simultaneous Multithreading (SMT), and to evaluate the improvement in Instructions Per Cycle (IPC) and functional-unit utilization.

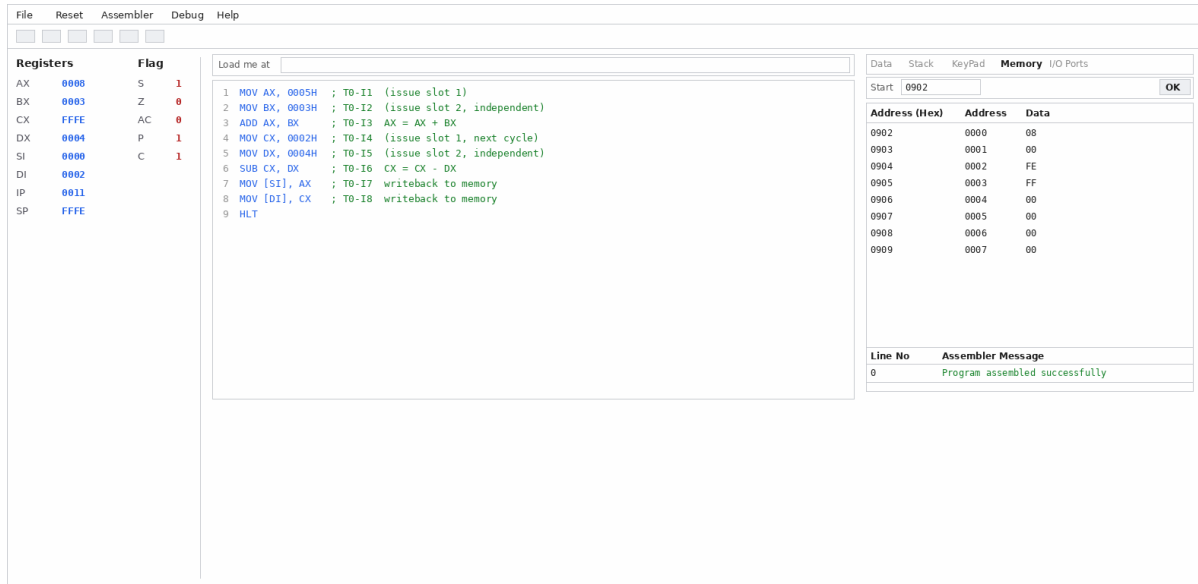
ALGORITHM:

1. Start the program and write an 8086 ALP containing independent instruction pairs suitable for dual-issue (superscalar width = 2).
2. Group the instructions into issue-pairs so that two independent instructions can be issued to separate execution units in the same cycle.
3. Reorder instructions (out-of-order execution) so independent instructions proceed while a dependent instruction waits in the reservation station/scoreboard.
4. Introduce a conditional branch and apply speculative execution: predict the outcome, continue execution speculatively, and roll back on misprediction.
5. Add a second independent instruction thread (T1) and interleave it with T0 so idle issue slots of the superscalar pipeline are filled (SMT).
6. Compute IPC for scalar (single-issue), superscalar (dual-issue), and superscalar + SMT execution.
7. Stop and compare functional-unit utilization across the three configurations.

CODE (8086 ALP - Thread T0, dual-issue groups):

```
MOV AX, 0005H ; T0-I1 (issue slot 1)
MOV BX, 0003H ; T0-I2 (issue slot 2, independent)
ADD AX, BX ; T0-I3 AX = AX + BX
MOV CX, 0002H ; T0-I4 (issue slot 1, next cycle)
MOV DX, 0004H ; T0-I5 (issue slot 2, independent)
SUB CX, DX ; T0-I6 CX = CX - DX
MOV [SI], AX ; T0-I7 writeback to memory
MOV [DI], CX ; T0-I8 writeback to memory
HLT
```

OUTPUT (Simulator Execution - Thread T0):



IPC & RESOURCE UTILIZATION COMPARISON:

Configuration	Instructions	Cycles	IPC
Scalar (single-issue)	8	8	1.0
Superscalar (2-wide, 1 stall cycle)	8	5	1.6
Superscalar + SMT (T0 + T1)	16	8	2.0

RESULT:

The 8086 instruction stream executed correctly on the simulator, and the calculated IPC increased from 1.0 for scalar execution to 1.6 for 2-way superscalar execution, and further to 2.0 when Simultaneous Multithreading filled otherwise idle issue slots with a second independent thread. This confirms that superscalar issue combined with out-of-order/speculative execution and SMT improves both instruction throughput and functional-unit utilization compared to a simple scalar pipeline.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation